Rails Security

Original Author: Yihao Xie, TA, CSCE 431 - Spring 2021, Fall 2021

Objectives

In this lab, we'll discuss three common security problems that can be prevented by developers to some extent.

- Weak Authentication
- XSS (Cross-Site Scripting)
- SQL Injections

Lab Contents

1. Weak Authentication

Potential damage: In general, most applications use a password mechanism of authentication, which means that users supply their user id's and passwords in an HTML form. But this is where problems might arise if your app uses a weak authentication mechanism. For example, an attacker might steal credentials, hijack accounts, take over cookies, or even hack your database where all your sensitive data is stored.

Rails doesn't come with a built-in authentication mechanism. However, developers can use several useful gems that provide all necessary authentication functions.

We will try Google OAuth in this Lab as the third-party authentication.
See primer here.

2. XSS (Cross-Site Scripting)

Potential damage: allows attackers to inject any type of client-side script which is then executed by the victims' browsers in their browsing contexts; to bypass access controls and to steal web sessions.

Ruby on Rails has a built-in XSS protection mechanism which automatically escapes all the data being transferred from Rails to HTML. While this is a big plus

for Rails framework security, it is not enough to solve all XSS problems, which is why every day new cross-site scripting vulnerabilities are still being discovered on Ruby on Rails web applications.

Fun practice:
A vulnerable web page https://examples.insecure.chefsecure.com/examples/script-injection
Try these attacks and see what will happen if your website is vulnerable to XSS.

```
1. <script>alert();</script>
2. <script>document.documentElement.innerHTML=''</script>
```

Let's learn more about XSS:

1) What is HTML Escaping
2) html_safe and Introduction to Safe Buffers
3) Transferring Data from Rails to HTML
4) Transferring Data from Rails to JavaScript
5) Transferring JSON Data to HTML/JavaScript
6) Insecure helper

3. SQL Injection

Potential Damage: SQL injection attacks aim at influencing database queries by manipulating web application parameters. A popular goal of SQL injection attacks is to bypass authorization. Another goal is to carry out data manipulation or reading arbitrary data

RailsGuides: "***Thanks to clever methods, this is hardly a problem in most Rails applications.*** *However, this is a very devastating and common attack in web applications, so it is important to understand the problem.*"

Let's learn by attacking
Example 1: https://www.hacksplaining.com/prevention/sql-injection

Example 2: https://medium.com/@sonalibhavsar977/ruby-on-rails-sql-injection-attack-and-prevention-9eb065f2ab67
Note:

1. In Example 2, you will clone a github repository. Fork it before you clone it.

2. Before you run bundle install, change the ruby version in Gemfile and .ruby-version. You also need to install a development dependency for mysql2
   $ sudo apt-get install libmysqlclient-dev, otherwise you won't be able to install gem mysql

Common attacks:
https://guides.rubyonrails.org/security.html#sql-injection

Prevention:
https://www.netsparker.com/blog/web-security/sql-injection-ruby-on-rails-development/

Recommended Reading:

This manual describes many common security problems in web applications and how to avoid them with Rails.
https://guides.rubyonrails.org/security.html

Other References:

1. https://rubygarage.org/blog/ruby-on-rails-web-application-vulnerabilities-how-to-make-your-app-secure (Open this link with your personal Google account if it doesn't work)
2. https://www.netsparker.com/blog/web-security/preventing-xss-ruby-on-rails-web-applications/